

packages/graph-view/scripts/benchmark-layout.mjs

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T19:24:46.865Z

Overview

A new script `packages/graph-view/scripts/benchmark-layout.mjs` has been added to benchmark Stage1 ELK layout performance.

Key changes

- **Imports:** `performance` from `node:perf_hooks` and `ELK` from `elkjs/lib/elk.bundled.js` (lines 16-17).
- **Constants:** `DEFAULT_NODE_WIDTH` (280) and `DEFAULT_NODE_HEIGHT` (120) (lines 19-21) to match `NODE_WIDTH/HEIGHT` in `src/utils/layout.ts`.
- **fillDims** (lines 30-37): recursively assigns default dimensions to nodes missing width/height, mirroring the production repair pass.
- **applyElkLayout** (lines 40-43): creates a repaired graph and runs `elk.layout`, exposing the same API as the production helper.
- **makeGraph** (lines 51-66): synthesizes a container-only graph with sparse edges; container count defaults to `Math.min(20, Math.ceil(nodeCount / 25))`.
- **bench** (lines 68-75): measures elapsed time for a single layout run and logs the result.
- **Execution:** script immediately benchmarks 500, 1000, and 3000 nodes (lines 78-80).

Impact

- No changes to runtime code; the script is isolated.
- Keeps benchmark logic close to the layout implementation, reducing duplication.
- Provides console output of Stage1 layout times, aiding regression detection.
- Requires Node ≥ 14 for `perf_hooks`; no browser impact.

Risks & follow-ups

- Verify that the bundled ELK version (`elkjs/lib/elk.bundled.js`) matches the production version; mismatches could skew results.
- Ensure `DEFAULT_NODE_WIDTH/HEIGHT` stay in sync with `src/utils/layout.ts`; drift would invalidate the benchmark.
- Confirm that `makeGraph` accurately reflects production container counts; otherwise, benchmark results may be misleading.
- Run the script in CI to capture baseline timings before future layout changes.